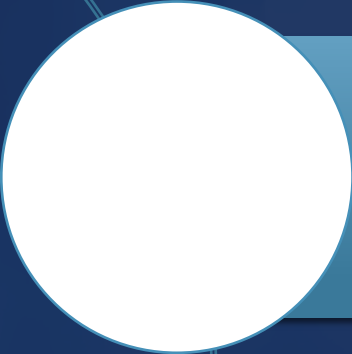




SISTEMAS OPERACIONAIS

AULA 4 – CHAMADA DE SISTEMAS E INTERRUPÇÃO

CONTEXTUALIZANDO



Aula passada:

- 1) Tipos de SOs
- 2) Formas de estruturar os SOs



Aula de hoje:

- 1) Chamadas de Sistemas
- 2) Interrupções

CHAMADAS DE SISTEMAS

Se uma aplicação precisa realizar alguma instrução privilegiada, por exemplo, imprimir um arquivo,

ela realiza uma **chamada de sistema**, que altera do modo usuário para o **modo kernel**, exemplo: ler um arquivo.

Chamadas de sistemas são, invariavelmente, a porta de entrada para o **modo kernel**.

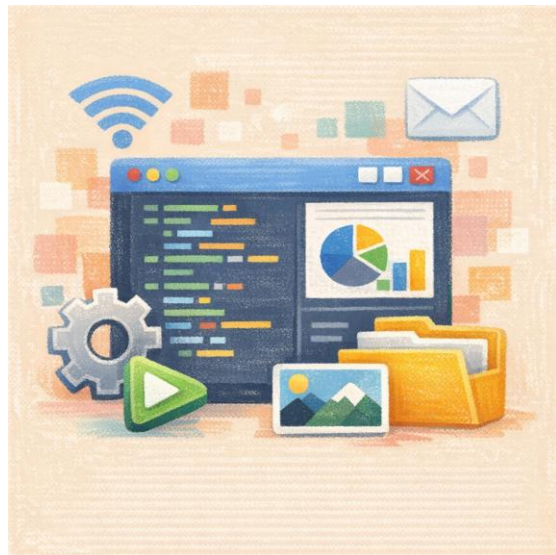
COMO ESTAS CHAMADAS SÃO REALIZADAS?

1) As chamadas de sistemas são realizadas através de instruções **TRAPS**.

2) TRAPS são conhecidos com **interrupções de software**.

3) Após o término da chamada, por exemplo, ler um arquivo, a execução **continua após a chamada de sistema**.

COMO ESTAS CHAMADAS SÃO REALIZADAS?



Programa do Usuário

SYSTEM CALL
(TRAP)

Salva contexto dos
registradores

Altera modo do
processador para kernel

Altera modo do
processador para usuário

Restaura o contexto dos
registradores

FUNÇÃO TRAP

Rotina do Sistema



SYSTEM CALL (TRAP) – OPEN()

Aplicação do Usuário

open ()

return

Modo usuário

Modo kernel

SYSTEM CALL INTERFACE



open ()
implementação
do método open ()
System call
*
*
*
return



INTERFACES DAS SYSCALLS

- Interface para esconder a complexidade das syscalls
- Interface de programação fornecida pelo SO
- Geralmente escrita em linguagem de alto nível (C, C++, Java ...)
- Normalmente as aplicações utilizam uma **Application Program Interface → API**
- Interface que encapsula o acesso direto às chamadas ao sistema

SYSCALL

Aplicação do Usuário

read(fd, buffer, var)

Modo usuário

Modo kernel

WRAPPER API - SysCall

read() do Solaris

read() do Linux

read() do AIX

SUN SOLARIS

LINUX

IBM AIX

INTERFACE DAS SYSCALLS

Interface das Chamadas de Sistemas (wrappers) mais utilizadas

WIN32 API para Windows

POSIX API para praticamente todas as versões de UNIX

JAVA API para a Java Virtual Machine JVM

INTERFACE DAS SYSCALLS

Motivos para utilizar **APIs** em vez das chamadas ao sistema

Portabilidade → independência da plataforma

Encapsulamento → esconder complexidade inerentes às chamadas ao sistema

Escalabilidade → acréscimo de funcionalidades que otimizam o desempenho

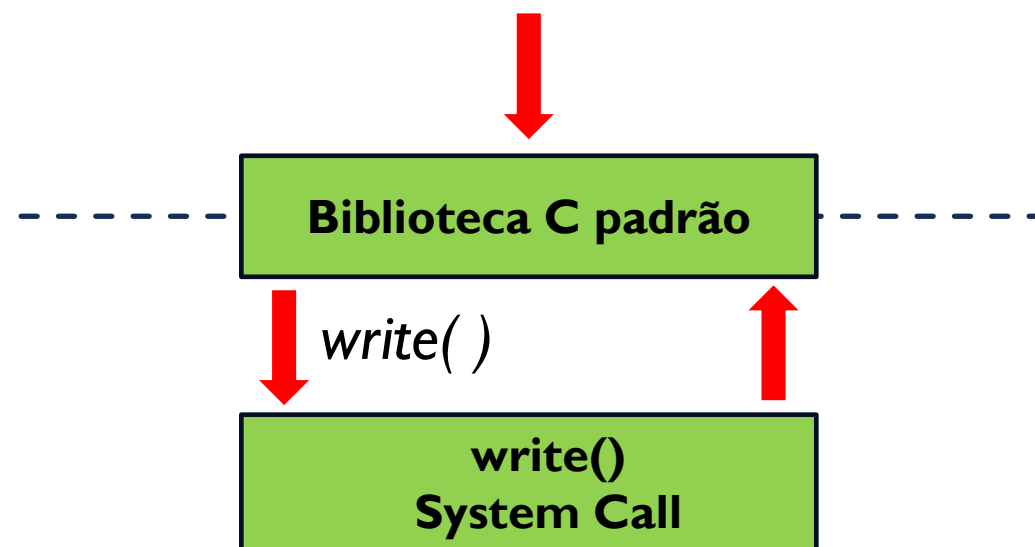
INTERFACE DAS SYSCALLS

Exemplo de uso com **printf()**

Programa em **C** que invoca a função de biblioteca **printf()** que por sua vez chama o system call **write()**

A chamada **printf()** ocasiona a chamada **write()** e **exit()**

```
# include <stdio.h>
Int main ( )
{
    printf("Olá, mundo!");
    *
    return 0;
}
```



INTERRUPÇÕES

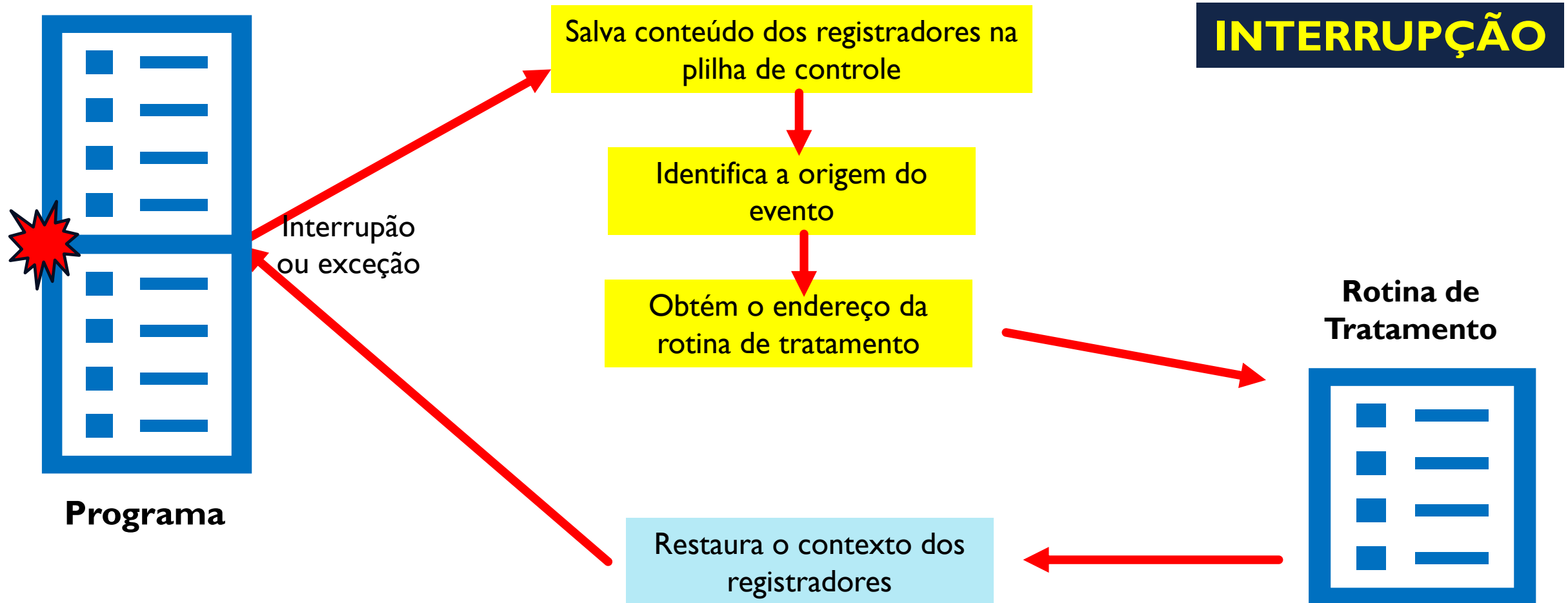
Vimos que um software pode interromper seu próprio processo (ao fazer uma chamada ao sistema):

1. Usando **TRAPS** (interrupções de software ou Exceções)
2. Para isso, a aplicação tem que estar rodando

Mas ocorrem **interrupções que não são causadas por apps**

1. Interrupções de hardware (**eventos externos**)
2. Um sinal elétrico no hardware
3. Causa: dispositivos de E/S ou o clock

COMO ESTAS CHAMADAS SÃO REALIZADAS?



INTERRUPÇÃO VERSUS TRAPS

Interrupção

Evento externo ao processador.

Gerada por dispositivos que precisam do SO.

Pode não estar relacionada ao processo que está rodando.

Traps

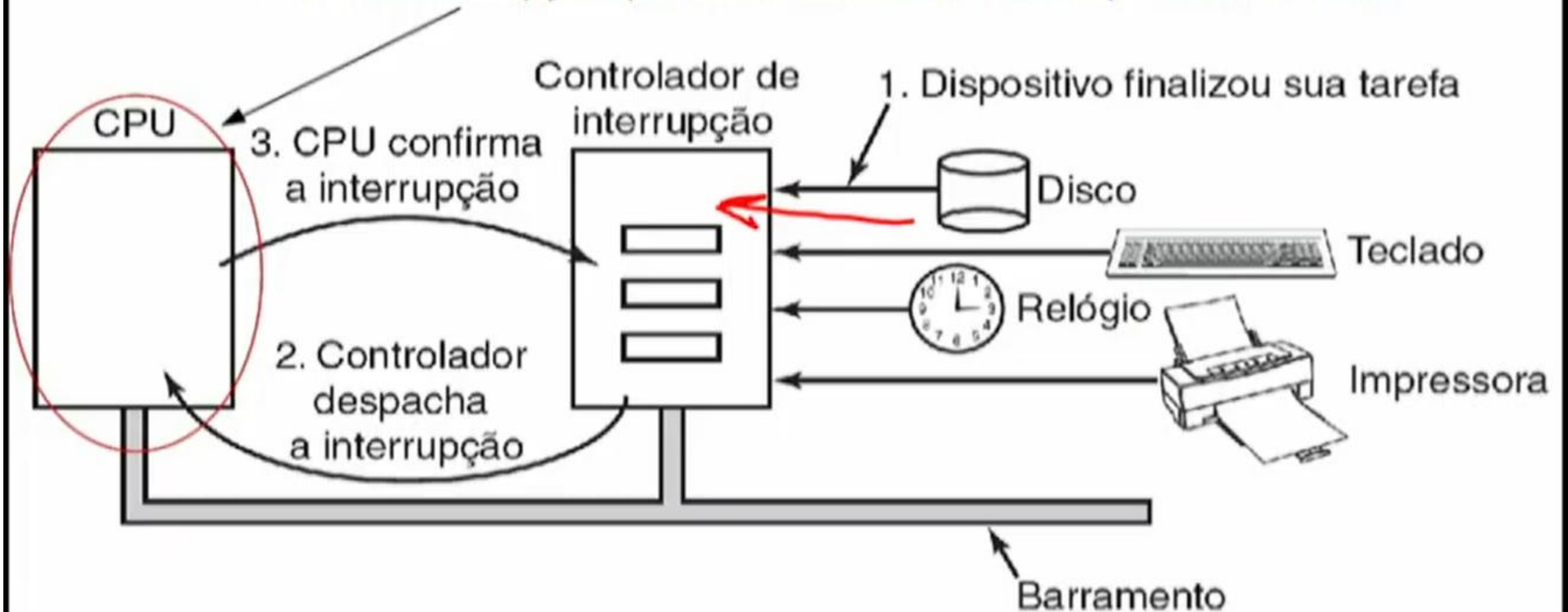
Evento inesperado **vindo de dentro do processador.**

Causados pelo processo corrente no processador
(seja por chamada ao SO, seja por interrupção ilegal).

EXEMPLO DE INTERRUPÇÃO

O novo contador de programa aponta para o início do procedimento de tratamento da interrupção.

Traps e interrupções usam o mesmo mecanismo a partir desse ponto
O vetor de interrupções pode estar tanto no hardware quanto na memória





PROFESSOR
PARISOTTO

www.parisotto.com.br